

CHAPTER 02

OpenCV 설치와 기초 사용법

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

2장 OpenCV 설치와 기초 사용법

2.1 OpenCV 개요와 설치

2.2 OpenCV 사용하기: HelloCV

2.2 OpenCV 사용하기: HelloCV

코드2-2: 영상을 화면에 출력하기

- 화면에 OpenCV 버전을 출력하고 영상파일(lenna.bmp)을 읽어서 화면에 출력해주는 프로그램
- 레나 (lenna) 영상은 영상 처리 및 컴퓨터 비전 분야에서 테스트용으로 널리 사용되는 영상임
- 간단한 예제를 통하여 OpenCV의 많이 사용하는 클래스와 함수들을 알아보자

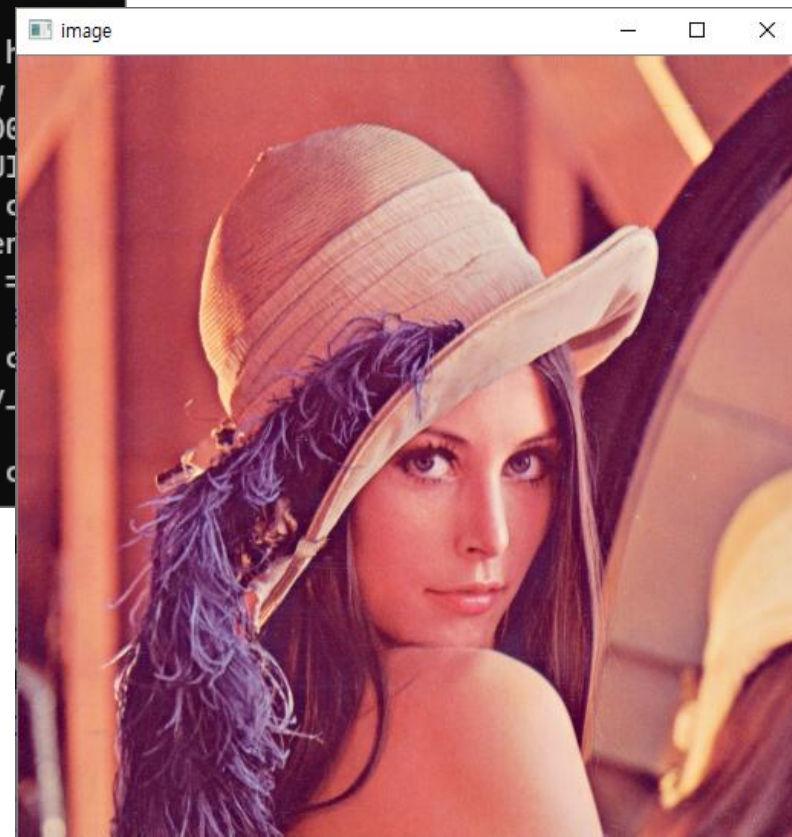
코드2-2 : 영상을 화면에 출력하기

```
#include "opencv2/opencv.hpp"
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    cout << "Hello OpenCV " << CV_VERSION << endl;
    Mat img;
    img = imread("lenna.bmp", IMREAD_COLOR);
    if ( img.empty( ) ) {
        cerr << "Image load failed!" << endl;
        return -1;
    }
    namedWindow("image");
    imshow("image", img);
    waitKey( );

    return 0;
}
```

코드2-2 : 영상을 화면에 출력하기

```
C:\Users\W2sungryul\sourceV x + - □ ×  
Hello OpenCV 4.10.0  
[ INFO:0@0.013] global registry.impl.hpp:114 cv::h  
gui_backend::UIBackendRegistry::UIBackendRegistry  
Enabled backends(4, sorted by priority): GTK(1000)  
GTK3(990); GTK2(980); WIN32(970) + BUILTIN(WIN32UI)  
[ INFO:0@0.015] global plugin_loader.impl.hpp:67 c  
plugin::impl::DynamicLib::libraryLoad load C:\oper  
build\x64\vc16\bin\opencv_highgui_gtk4100_64.dll =  
FAILED  
[ INFO:0@0.016] global plugin_loader.impl.hpp:67 c  
plugin::impl::DynamicLib::libraryLoad load opencv_  
hgui_gtk4100_64.dll => FAILED  
[ INFO:0@0.017] global plugin_loader.impl.hpp:67 c
```



헤더파일 포함하기

- opencv.hpp 파일에서 모든 헤더파일을 포함하고 있음

[opencv.hpp 헤더 파일 내용]

```
#ifndef __OPENCV_ALL_HPP__
#define __OPENCV_ALL_HPP__

#include "opencv2/core.hpp"
#include "opencv2/imgproc.hpp"
#include "opencv2/photo.hpp"
#include "opencv2/video.hpp"
#include "opencv2/features2d.hpp"
#include "opencv2/objdetect.hpp"
#include "opencv2/calib3d.hpp"
#include "opencv2/imgcodecs.hpp"
#include "opencv2/videoio.hpp"
#include "opencv2/highgui.hpp"
#include "opencv2/ml.hpp"

#endif
```

네임스페이스 cv

- OpenCV에서 정의된 모든 클래스와 함수는 cv 네임스페이스 안에 선언되어 있음
- 모든 코드작성시 `using namespace cv;` 코드를 추가하여 `cv::`를 생략할 수 있다.

Mat 클래스

- 영상데이터를 저장할 때 사용하는 클래스, 자세한 내용은 3장 참고
- Mat 객체를 생성하고 imread 함수를 이용하여 파일에서 영상을 불러와서 Mat 객체에 저장함

```
#include <opencv2/opencv.hpp>
...
    Mat img;
    img = imread("lenna.bmp", IMREAD_COLOR);
...
```

cerr 객체와 에러처리

- 함수실행시 반드시 실행결과를 확인하여 에러발생시는 사용자가 알 수 있도록 에러메시지를 출력하고 프로그램을 종료해야 한다.
- cerr : 에러 메시지를 출력할때 사용하는 출력스트림 객체, cout 객체와 거의 같음
- 메인 함수의 리턴값이 0이면 정상종료를 의미하고 리턴값이 -1이면 비정상 종료를 의미, 메인 함수의 리턴값은 운영체제가 받아서 처리

```
Mat img;
img = imread("lenna.bmp", IMREAD_COLOR);
if ( img.empty( ) ) {
    cerr << "Image load failed!" << endl;
    return -1;
}
```

imread 함수

```
Mat imread(const String& filename, int flags = IMREAD_COLOR);
```

- **filename** 불러올 영상 파일 이름
- **flags** 영상 파일 불러오기 옵션 플래그, ImreadModes 열거형 상수를 지정합니다.
- **반환값** 불러온 영상 데이터(Mat 객체)

- 매개변수 filename의 자료형 **String**은 std::string의 이름 재정의임 -> typedef std::string **cv::String**
- filename에 "lenna.bmp"처럼 파일 이름만 지정하면 프로젝트 폴더에 위치한 lenna.bmp 파일을 불러옴
- BMP, JPG, TIF, PNG 등 대부분의 영상 파일 형식을 지원
- 만약 다른 폴더의 파일을 불러오려면 절대 경로 형식으로 지정함
 - C:\Users\WW\doc\lenna.bmp -> Windows 스타일
 - C:/users/doc/lenna.bmp -> Linux 스타일

imread 함수

- flags : 영상 파일을 불러올 때 사용할 컬러모드와 영상 크기를 지정하는 플래그임, ImreadModes 열거형 상수 중의 하나로 지정

표 2-4 주요 ImreadModes 열거형 상수

ImreadModes 열거형 상수	설명
IMREAD_UNCHANGED	입력 파일에 지정된 그대로의 컬러 속성을 사용합니다. 투명한 PNG 또는 TIFF 파일의 경우, 알파 채널까지 이용하여 4채널 영상으로 불러옵니다.
IMREAD_GRAYSCALE	1채널 그레이스케일 영상으로 변환하여 불러옵니다.
IMREAD_COLOR	3채널 BGR 컬러 영상으로 변환하여 불러옵니다.
IMREAD_REDUCED_GRAYSCALE_2	크기를 1/2로 줄인 1채널 그레이스케일 영상으로 변환합니다.
IMREAD_REDUCED_COLOR_2	크기를 1/2로 줄인 3채널 BGR 영상으로 변환합니다.
IMREAD_IGNORE_ORIENTATION	EXIF에 저장된 방향 정보를 사용하지 않습니다.

imread 함수

- flags 인자는 기본값으로 IMREAD_COLOR가 지정됨 -> 호출 시 두 번째 인자를 지정하지 않으면 자동으로 3채널 컬러 영상 형식으로 영상을 불러옴

```
imread("lenna.bmp"); -> imread("lenna.bmp", IMREAD_COLOR);
```

- flags 인자를 IMREAD_GRAYSCALE으로 지정하면 영상파일을 1채널 그레이스케일 영상으로 불러옴

```
imread("lenna.bmp", IMREAD_GRAYSCALE);
```

- imread() 함수는 filename 영상 파일을 불러와 Mat 객체로 변환하여 반환함

Mat::empty 함수

```
bool Mat::empty() const
```

• 반환값 행렬의 rows 또는 cols 멤버 변수가 0이거나, 또는 data 멤버 변수가 NULL이면 true를 반환합니다.

- Mat::empty() 함수는 Mat 클래스의 멤버 함수
- imread 함수를 이용하여 lenna.bmp 파일을 불러온 후, 영상 데이터가 정상적으로 불러왔는지를 확인하기 위해 사용함
- 에러발생시 true, 정상이면 false를 리턴, 에러시는 반드시 에러메시지 출력 후 프로그램을 종료해야 함

```
img = imread("lenna.bmp", IMREAD_COLOR);  
if ( img.empty( ) ) {  
    cerr << "Image load failed!" << endl;  
    return -1;  
}
```

imwrite 함수

```
bool imwrite(const String& filename, InputArray img,
             const std::vector<int>& params = std::vector<int>());
```

- **filename** 저장할 영상 파일 이름
- **img** 저장할 영상 데이터(Mat 객체)
- **params** 저장할 영상 파일 형식에 의존적인 파라미터(플래그 & 값) 쌍
(paramId_1, paramValue_1, paramId_2, paramValue_2, ...)
- **반환값** 정상적으로 저장하면 true, 실패하면 false를 반환합니다.

- Mat 객체 img 변수에 저장되어 있는 영상 데이터를 filename 이름의 파일로 저장함, 저장위치는 프로젝트 폴더임

```
imwrite("myfile.jpg", img);
```

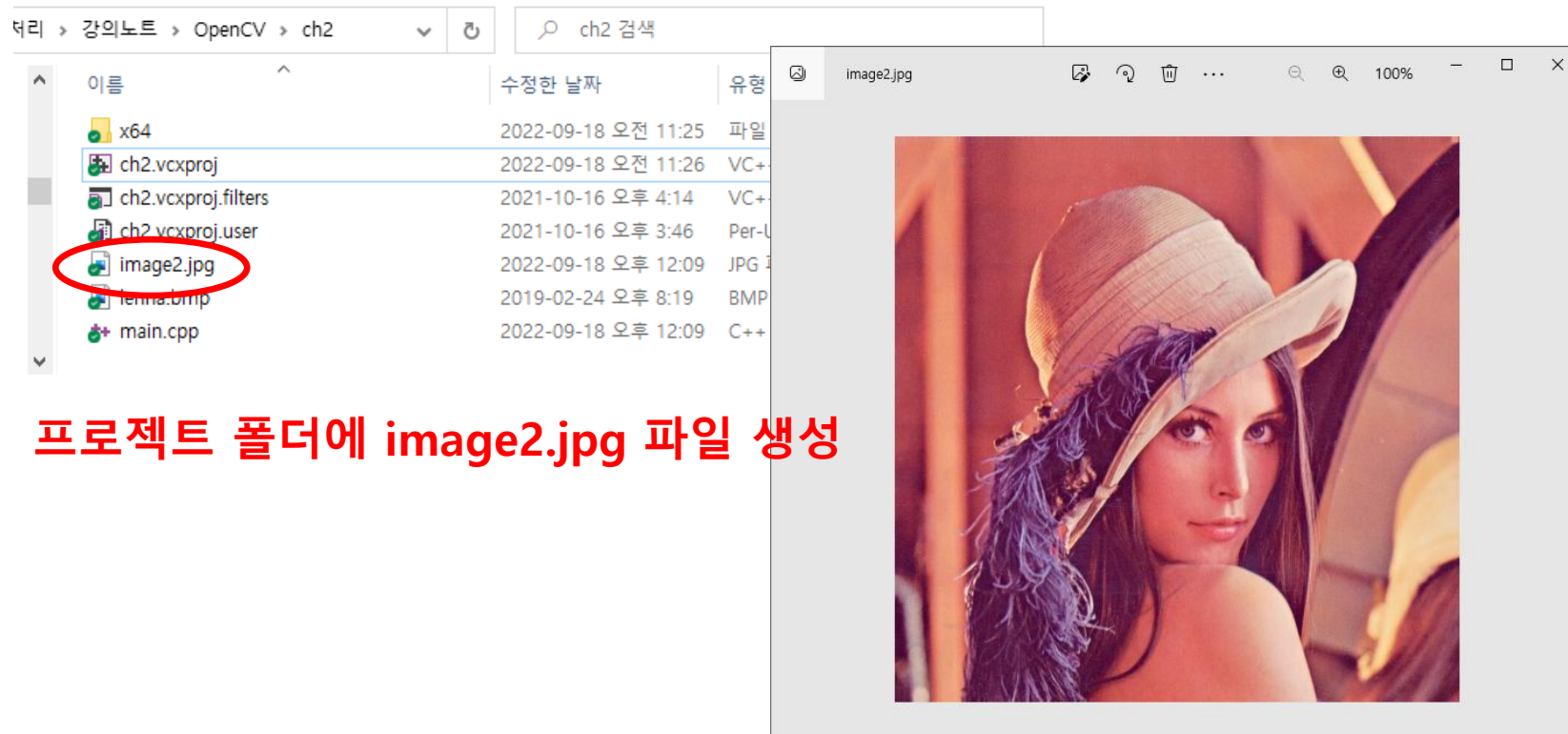
- 영상 파일 형식은 filename 문자열의 파일 확장자에 의해 결정됨
- params 인자에는 저장할 파일 형식에 의존적인 별도의 옵션을 지정할 수 있음, 자세한 내용은 Opencv 도움말 사이트 참고

코드2-3 : imwrite함수 예제

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img;
    img = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1;}

    imshow("image", img);
    imwrite("image2.jpg", img); // img객체를 image2.jpg 파일로 저장
    waitKey();
    return 0;
}
```


코드2-3 : imwrite함수 예제



프로젝트 폴더에 image2.jpg 파일 생성

namedWindow 함수

```
void namedWindow(const String& winname, int flags = WINDOW_AUTOSIZE);
```

- **winname** 영상 출력 창 상단에 출력되는 창 고유 이름. 이 문자열로 창을 구분합니다.
- **flags** 생성되는 창의 속성을 지정하는 플래그. WindowFlags 열거형 상수를 지정합니다.

- 영상 출력을 위한 빈 윈도우(창)를 생성하는 함수임
- 두 개의 인자로 구성되어 있지만, 두 번째 인자는 기본 인자가 있으므로 winname 문자열 하나만 지정하여 사용할 수 있음
- 생성된 여러 개의 윈도우(창)를 구분하기 위해 OpenCV에서는 각각의 창에 고유한 문자열 winname을 부여하여 각각의 창을 구분함
- 새로운 창을 만들 때에는 winname 인자에 고유한 문자열을 지정해야 함

namedWindow 함수

- winname으로 지정한 창의 고유 이름은 실제 생성되는 창의 상단 제목 표시줄에 출력됨
- namedWindow() 함수의 두 번째 인자 flags는 새로 생성하는 창의 속성을 지정하는 용도로 사용됨

표 2-5 주요 WindowFlags 열거형 상수

WindowFlags 열거형 상수	설명
WINDOW_NORMAL	영상 출력 창의 크기에 맞게 영상 크기가 변경되어 출력됩니다. 사용자가 자유롭게 창 크기를 변경할 수 있습니다.
WINDOW_AUTOSIZE	출력하는 영상 크기에 맞게 창 크기가 자동으로 변경됩니다. 사용자가 임의로 창 크기를 변경할 수 없습니다.
WINDOW_OPENGL	OpenGL을 지원합니다.

imshow 함수

```
void imshow(const String& winname, InputArray mat);
```

- **winname** 영상을 출력할 대상 창 이름
- **mat** 출력할 영상 데이터(Mat 객체)

- Mat 클래스 객체에 저장된 영상 데이터를 winname 이름의 윈도우(창)에 출력
- 호출시 mat 매개변수에 Mat 객체를 전달하면 됨
- InputArray 타입은 Mat, vector<T> 등 다양한 객체를 표현할 수 있는 인터페이스 클래스이며, 주로 OpenCV 함수 입력에 해당하는 인자의 자료형으로 사용됨
- OpenCV 함수 설명에서 인자 타입이 InputArray라고 되어 있으면 대부분 Mat 클래스 타입의 변수를 전달한다고 생각하여도 무방함

imshow 함수

- mat 객체에 저장된 영상이 1채널 8비트 uchar 자료형으로 구성된 그레이스케일 영상이라면 픽셀 값을 그대로 그레이스케일 밝기 형태로 나타냄
- mat 객체에 저장된 영상이 uchar 자료형을 사용하는 3채널 컬러 영상이라면 색상 채널이 파란색(Blue), 녹색(Green), 빨간색(Red) 순서로 되어 있다고 간주하여 색상을 표현함
- 만약 mat 객체가 부호 없는 16비트 또는 32비트 정수형이라면 행렬 원소 값을 256으로 나눈 값을 영상의 밝기 값으로 사용함
- 반면에 mat 객체가 32비트 또는 64비트 실수형 행렬이라면 행렬 원소에 255를 곱한 값을 밝기 값으로 사용함

waitKey 함수

```
int waitKey(int delay = 0);
```

- **delay** 키 입력을 기다릴 시간(밀리초 단위). $\text{delay} \leq 0$ 이면 무한히 기다립니다.
- **반환값** 눌린 키 값. 지정한 시간 동안 키가 눌리지 않았으면 -1을 반환합니다.

- delay 밀리초 동안 키보드 입력을 기다림
- delay 밀리초 동안 키보드 입력이 들어오면 바로 리턴하고 리턴값은 입력된 키의 아스키코드임
- delay 밀리초 동안 키보드 입력이 들어오지 않으면 지정된 시간동안 기다리다가 -1을 리턴함
- delay = 0이면 키입력이 들어올 때까지 무한히 대기한다.
- **imshow 함수 호출 후 waitKey 함수를 호출해야 영상이 화면에 출력됨, imshow 함수 다음에 반드시 waitKey 함수를 호출할 것.**
- **키보드 입력을 받을 때는 영상창이 선택된 상태에서 입력해야 함**

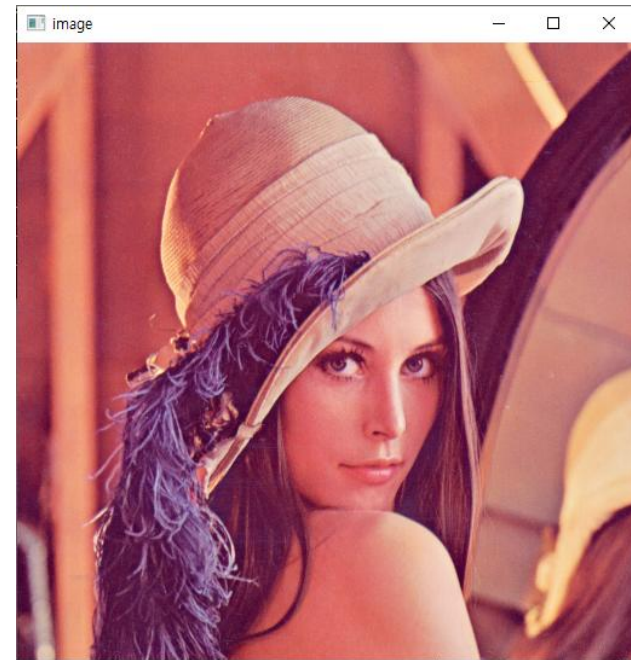
코드2-4 : waitKey함수 예제

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    int count = 0;
    Mat img;
    img = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1;}
    imshow("image", img);
    while (true)
    {
        cout << count++ << "초, 종료하려면 q를 누르세요" << endl;
        int ch = waitKey(1000); // 1000msec 동안 지연됨
        if (ch == 'q') break;    // 키보드에서 q가 입력되면 종료
    }
    return 0;
}
```

코드2-4 : waitKey함수 예제

```
Microsoft Visual Studio 디버그 콘솔
0초, 종료하려면 아무 키나 누르세요
1초, 종료하려면 아무 키나 누르세요
2초, 종료하려면 아무 키나 누르세요
3초, 종료하려면 아무 키나 누르세요
4초, 종료하려면 아무 키나 누르세요
5초, 종료하려면 아무 키나 누르세요
6초, 종료하려면 아무 키나 누르세요
7초, 종료하려면 아무 키나 누르세요
8초, 종료하려면 아무 키나 누르세요

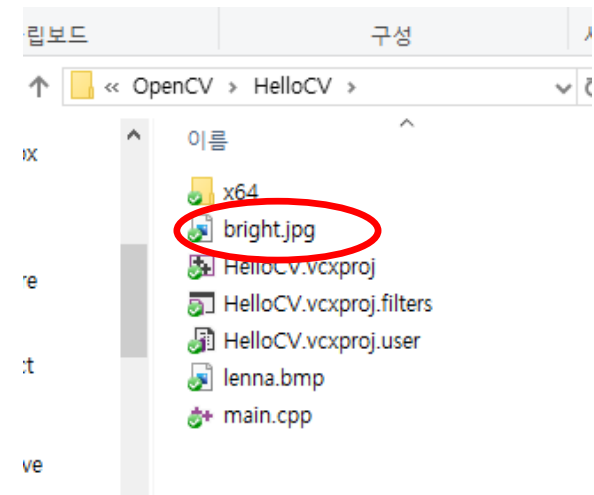
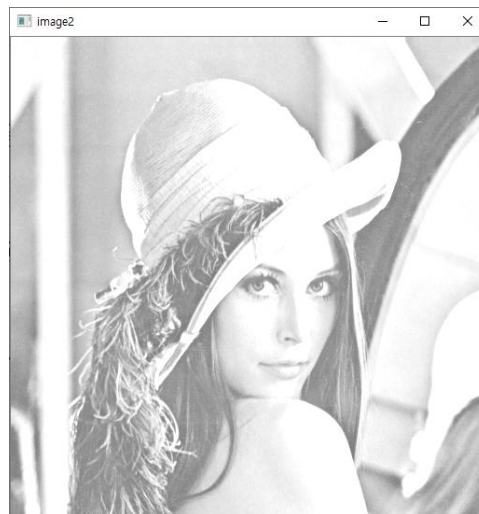
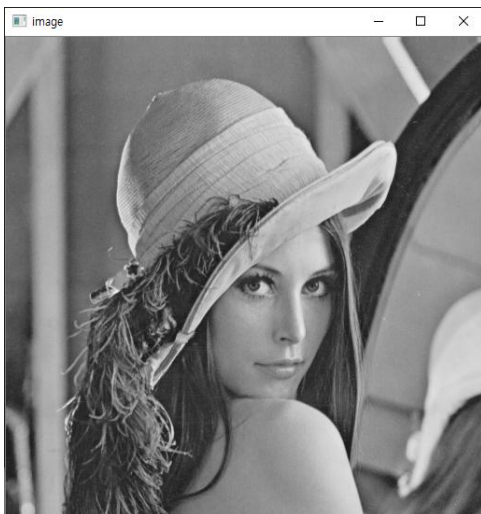
D:\Users\sungryul\Dropbox\Lecture\2022년2학기\영상처리\강의노트\OpenCV\wx64\Debug\ch2.exe(프로세스 3620개)이(가) 종료되었습니다(코드 : 0개).
이 창을 닫으려면 아무 키나 누르세요...
```



- *키보드 입력을 받을 때는 영상창이 선택된 상태에서 입력해야 함
- *콘솔창이 선택된 상태에서는 입력 못 받음

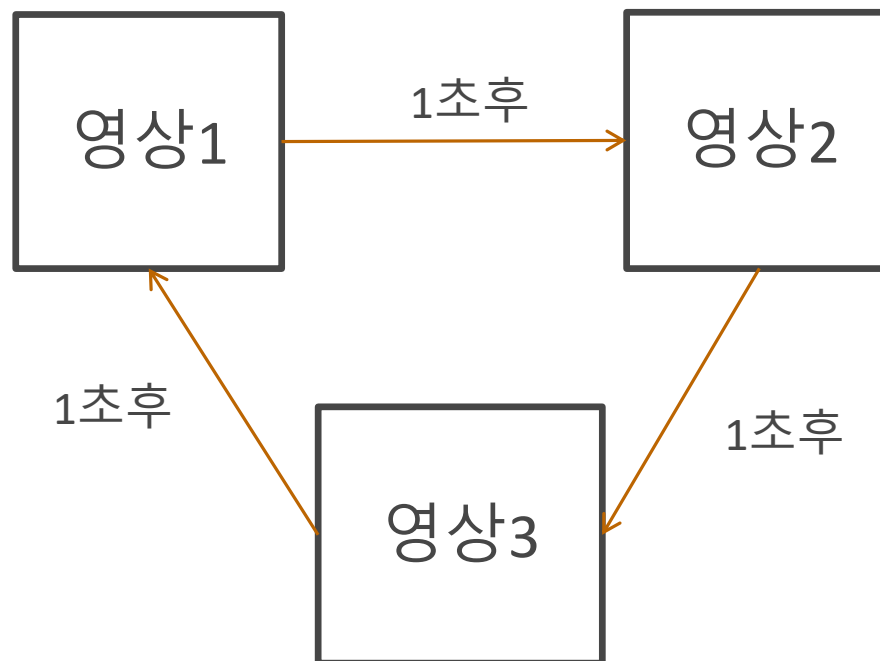
과제1

- 예제 코드2-2에서 다음 알고리즘대로 코드를 수정하고 저장한 결과파일(bright.jpg)을 그림판에서 열어서 확인하라.
- 알고리즘
 1. lenna.bmp 파일을 그레이스케일 영상으로 읽어서 화면에 출력
 2. 영상의 밝기를 증가시킴 : $img = img + 100$;
 3. 새로운 윈도우를 생성하여 밝아진 영상을 화면에 출력
 4. 밝아진 영상을 bright.jpg 파일로 저장(imwrite함수 사용)



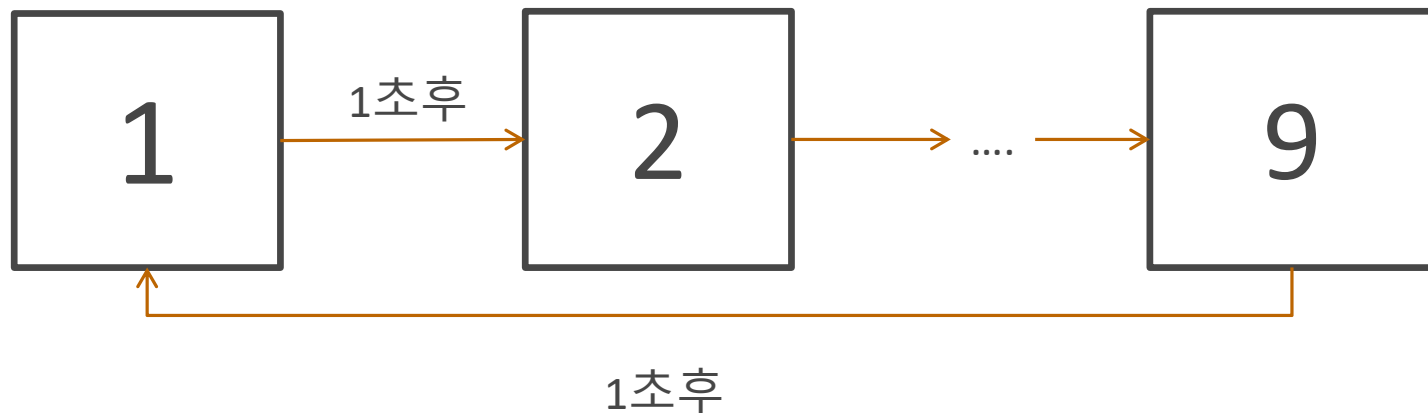
과제2

- 인터넷으로부터 3개의 영상을 다운받아 1초 간격으로 하나의 윈도우에 3개의 영상을 번갈아 출력해주는 프로그램을 작성하시오.
- Mat 객체를 생성하고 영상을 불러온 후 waitKey함수를 이용하여 출력시간을 조절하고 배열과 반복문으로 코드를 최대한 짧게 작성하라. waitKey함수는 1회만 호출하라.
- 실행결과를 동영상으로 저장하여 제출할 것



과제3

- 과제2를 이용하여 0부터 9까지 카운트하는 초시계를 만들어 보라.
- 그림판을 이용하여 0~9 숫자영상을 만들고 1초마다 번갈아 같은 창에 출력해준다.
- 배열과 반복문을 이용하여 코드를 최대한 짧게 작성하라.
waitKey함수는 1회만 호출하라.
- 숫자영상의 크기와 숫자의 위치는 동일하게 만든다.
- 실행결과를 동영상으로 저장하여 제출할 것



과제 제출 방법

- 소스파일, 라인단위의 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.